

Continuous Integration and Development

Move fast and break nothing

Presented by

Dr. Huaming Chen

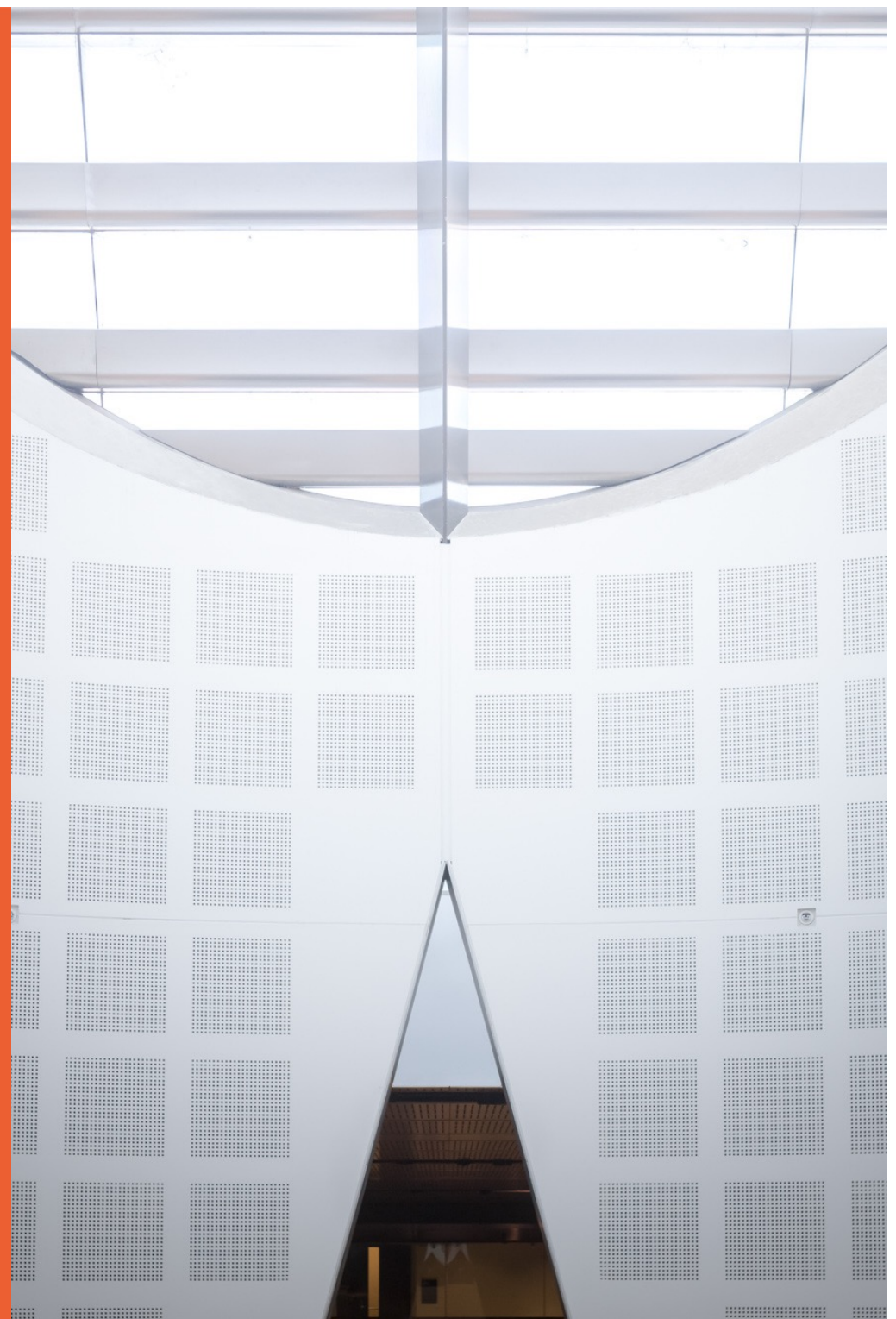
ELEC3609/9609

Internet Software Platforms

2025 S2



THE UNIVERSITY OF
SYDNEY



We acknowledge the tradition of
custodianship and law of the Country on which
the University of Sydney campuses stand.
We pay our respects to those who have cared
and continue to care for Country.

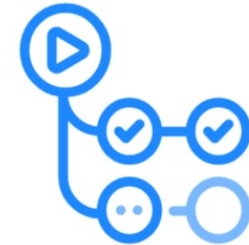


THE UNIVERSITY OF
SYDNEY

Overview

- ✓ **Continuous Integration**
 - ✓ **Integration Hell**
 - ✓ **Goals**
 - ✓ **Considerations**
 - ✓ **Conflicts**
 - ✓ **Linting and Static Analysis**
 - ✓ **Building**
 - ✓ **Testing**
 - ✓ **Publishing**
- ✓ **Release and Deployment**
- ✓ **Post-Deployment**

django



Continuous Integration

What is Continuous Integration?

Long ago, in the dark times, teams of developers would each work on their own features until it was time to integrate into a coherent product.

In the Integration Phase, the changes from tens or hundreds of developers would be merged into a common code base which would fail to compile due to conflicts.

And so, the developers would enter into Integration Hell. Sometimes they would emerge victorious. Other times the project would fail causing much weeping and gnashing of teeth.

Integration Hell

- A state where new changes cannot be integrated into the mainline codebase quickly or effectively. This can occur because:
 - The changes have deviated too much from master. Solution: Rebase often.
 - The change itself is too large and touches too many files.

Solution: Ensure developers are scoping their changes correctly. Prefer small changes over big changes.

- We want to have confidence in our changes and spend our limited development time building new features, not integrating existing work into a stable product or service.

What is Continuous Integration?

- Continuous integration (CI) is the practice of merging all developer's working copies to a shared mainline codebase several times a day.

Goals

- Avoid Integration Hell, a state in which developers spend more time checking in code to the repository than writing the code itself.

Goals

- Avoid Integration Hell, a state in which developers spend more time checking in code to the repository than writing the code itself.
- Provide an isolated, reproducible test environment.

Goals

- Avoid Integration Hell, a state in which developers spend more time checking in code to the repository than writing the code itself.
- Provide an isolated, reproducible test environment.
- Provide an isolated, reproducible build environment.

```
pip install -r requirements.txt  
python manage.py collectstatic --noinput  
npm ci && npm run build
```

Goals

- Avoid Integration Hell, a state in which developers spend more time checking in code to the repository than writing the code itself.
- Provide an isolated, reproducible test environment.
- Provide an isolated, reproducible build environment.
- Increase deployment confidence.

Goals

- Avoid Integration Hell, a state in which developers spend more time checking in code to the repository than writing the code itself.
- Provide an isolated, reproducible test environment.
- Provide an isolated, reproducible build environment.
- Increase deployment confidence.
- Enforce coding style and code quality.



```
pip install black flake8
```

```
npm install --save-dev prettier eslint stylelint
```

```
- run: black --check .
```

```
- run: flake8
```

```
- run: npx prettier --check .
```

Goals

- Avoid Integration Hell, a state in which developers spend more time checking in code to the repository than writing the code itself.
- Provide an isolated, reproducible test environment.
- Provide an isolated, reproducible build environment.
- Increase deployment confidence.
- Enforce coding style and code quality.
- Test code in multiple environments.

Goals

- Avoid Integration Hell, a state in which developers spend more time checking in code to the repository than writing the code itself.
- Provide an isolated, reproducible test environment.
- Provide an isolated, reproducible build environment.
- Increase deployment confidence.
- Enforce coding style and code quality.
- Test code in multiple environments.
- Streamline development workflows and increase productivity.

Considerations

- Speed
- Reproducibility
- Reliability
- Clarity

Conflicts

- By encouraging small, incremental changes, the adoption of Continuous Integration aims to minimise the cost associated with resolving conflicts.
- There are different ways in which an application can fail during/after integration, and these are all “conflicts”.

Conflicts

- By encouraging small, incremental changes, the adoption of Continuous Integration aims to minimize the cost associated with resolving conflicts.
- There are different ways in which an application can fail during/after integration, and these are all “conflicts”.
 - **Merge conflict:** Developers have modified the same file concurrently.

Conflicts

- By encouraging small, incremental changes, the adoption of Continuous Integration aims to minimize the cost associated with resolving conflicts.
- There are different ways in which an application can fail during/after integration, and these are all “conflicts”.
 - **Merge conflict:** Developers have modified the same file concurrently.
 - **Functional conflict:** Developers have modified files concurrently such that the resulting system does not compile/build.

Conflicts

- By encouraging small, incremental changes, the adoption of Continuous Integration aims to minimize the cost associated with resolving conflicts.
- There are different ways in which an application can fail during/after integration, and these are all “conflicts”.
 - **Merge conflict:** Developers have modified the same file concurrently.
 - **Functional conflict:** Developers have modified files concurrently such that the resulting system does not compile/build.
 - **Logical conflict:** Developers have modified files concurrently such that the system does not execute successfully.

```
# auth.py
def logout(user):
    user.token = None
```

```
# Fixed logout function
def logout(user, timeout=False):
    if timeout:
        user.token = None
```

Conflicts

- The best way to minimize the cost associated with resolving conflicts is to avoid conflicts altogether.
 - This is difficult to achieve.
 - Ideally, components of an application would be independent with very few dependencies.
 - Practically, it makes sense to write reusable modules that are shared between components.
 - Even with well-isolated components, there has to be some process which builds all of the separate modules into a single application (this is called “integration”).
- Naturally, the more developers that are committing to a code base, the higher the risk of conflicts.

Stale Branches

- There is a general expectation that HEAD (usually the master branch) is always in a functional state. As such, conflicts are expected to be resolved by the developer that is last to land their changes.
- The longer a branch of code remains checked out, the greater the risk of multiple integration conflicts and failures when the developer branch is reintegrated into the main line.
- When developers submit code to the repository, they must first update (merge or rebase) their code to reflect the changes in the repository since they took their copy. The more changes the repository contains, the more work developers must do before submitting their own changes.

Stale Branches

- Rebasing your changes onto master is not fun.
- Test results may no longer be valid after the change has been merged/rebased.
- Eventually, the repository may become so different from the developers' baselines that the time it takes to integrate exceeds the time it took to make their original changes.
- In a worst-case scenario, developers may have to discard their changes and completely redo the work as the original purpose of the changes could now be invalid or require a completely different approach.

Benefits of Small Incremental Changes

When making incremental changes:

- Integration bugs are detected early and are easy to track down due to small change sets. This saves both time and money over the lifespan of a project.
- When unit tests fail or a bug emerges, if developers need to revert the codebase to a bug-free state without debugging, only a small number of changes are lost (because integration happens frequently).

What Does Continuous Integration Actually Do?

- Linting and static analysis
- Building
- Testing
- Unit testing
- Integration testing
- Acceptance testing
- Regression testing
- Publishing, Releasing and Deploying

Linting and Static Analysis

- **Linting** is the process of running a program that will analyze code for potential errors.
- Lint was the name originally given to a particular program that flagged some suspicious and non-portable constructs (likely to be bugs) in C language source code.
- It can also enforce any code style guides.

Linting and Static Analysis

- **Static analysis**, also called static code analysis, is a method of computer program debugging that is done by examining the code without executing the program.
- The process provides an understanding of the code structure and can help to ensure that the code adheres to industry standards.

Linting vs Static Analysis

- Semantically, linting and static analysis are very similar tools.
- Generally, however, linting refers to a subset of static analysis, which consists only of rules which are fast and accurate.

Linting vs Static Analysis

Linting

- Zero false positives
- Violations can often be automatically fixed.
- Performed on a file-by-file basis.
- Fast, rapid feedback.

Static Analysis

- Some tolerance for false positives
- Violations cannot usually be auto-fixed.
- Generally, analyses the entire code base.
- Slower, but comprehensive

```
def calculate_sum(a, b):  
    result = a + b  
    return a
```

Linting vs Static Analysis

Linting

- Coding style
- Deprecated classes/functions
- Inefficient language constructs
- Naming conventions
- Spelling mistakes
- Syntax errors
- Unreachable code
- Unused variables

Static Analysis

- Cyclomatic complexity
- Type inference
- Npath complexity
- Unused classes/functions

Building

- Traditionally “building” refers to compilation, the translation of source code into object code.
- However, dynamic languages (PHP, Python, Ruby) do not require compilation.

Building

- In the context of CI, building **backend** code usually refers to more than just compilation:
 - Optimising the autoloader (if applicable) or symbol map
 - Pre-populating caches
 - Pre-rendering templates
 - Vendoring dependencies
- Building **frontend** code refers to a different process:
 - Bundling CSS/JavaScript assets
 - Minifying images (lossy vs lossless compression)
 - Minifying CSS/JavaScript assets (obfuscation, whitespace)
 - Transpiling JavaScript (source-to-source compilation)
 - Vendoring dependencies

Unit Testing

- Language specific tooling:
 - Golang: go test
 - Java: JUnit
 - Python: Nosetests/Pytest
 - PHP: PHPUnit
 - Ruby: RSpec
- Basic criteria:
 - Fast to execute
 - One-to-one mapping with source code
 - Zero or minimal dependencies (other code, filesystem, network, etc)
 - White box testing
- Can be executed in different environments to help future-proof your application.

Integration Testing

- Ensures that different components of your application interact in the expected manner.
- Should be executed after unit tests.
- Should focus on testing the API rather than implementation details.
- Black box testing.

Acceptance Testing

- Focuses on functionality from a user perspective.
- Tests the application from end-to-end.
- Very slow (relatively), but very important.
- Cross-browser testing.

Regression Testing

- Used to prevent bugs from reoccurring.
- When fixing a bug, consider adding a regression test.
- Regression tests often cover bizarre edge cases.

Publishing

- At the completion of a CI job, you would typically produce one or more artifacts.
- The artifacts produced depend on the programming language and tooling that is being used, as well as the deployment process.
- Examples of artifacts:
 - Archive (tar, zip)
 - Binary
 - Mobile application (apk)
 - Package (deb, rpm)
- Publishing is a process which exposes artifacts from CI to the deployment process.

Automatic Deployment

- If you are feeling brave, you can trigger deployments automatically at the completion of your CI job.
- This is referred to as “Continuous Deployment”.

Release and Deployment

What is Deployment?

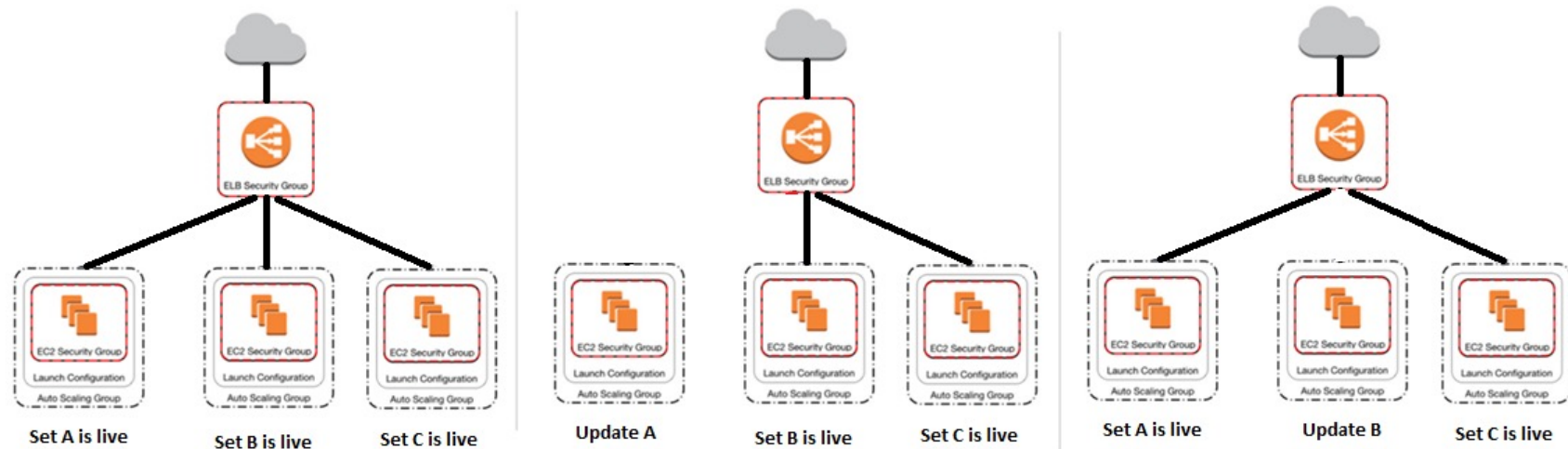
- Deployment is the process by which the latest version of an application is delivered to the users.
- Examples include:
 - Submitting an APK to Google Play Store
 - Updating code on a fleet of production server
 - Pushing files to an S3 bucket

How to Deploy?

- How often to deploy?
- Use existing infrastructure or new?
- Automated/manual?
- Monitoring
- Rolling back

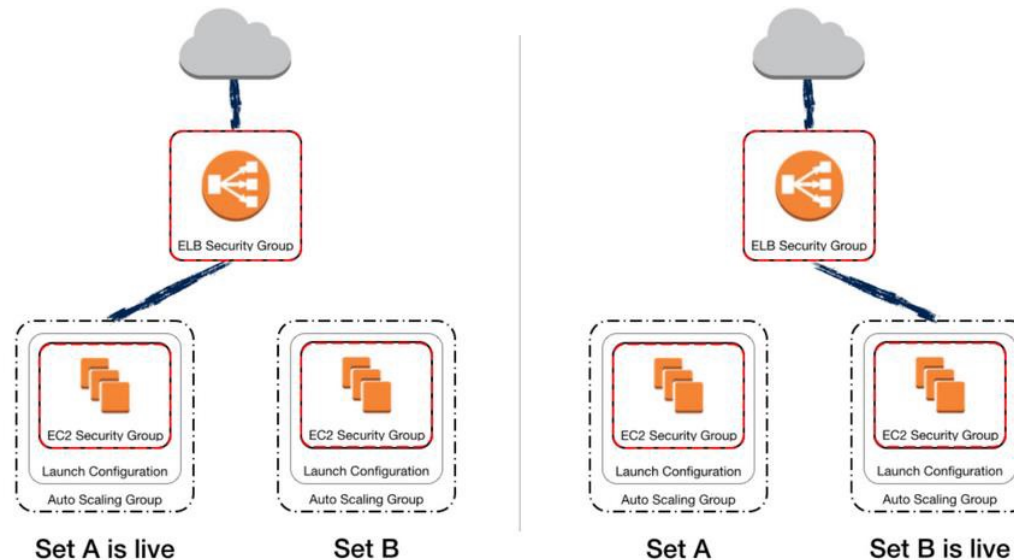
Types of Deployment?

- Rolling deployment
 - Progressively deploy/release the new version until the entire pool has been updated.
 - Will serve a mix of old and new assets.



Types of Deployment

- Blue-green deployment
 - Spin up an entirely new set of infra with the new code.
 - Do a hard switch over of all traffic to new infra/code



Post Deployment

Post Deployment

- Now that our changes are in the hands of users, how do we verify that everything is working as expected?
- As covered in previous week (Week 6), now is the time to look at:
 - Testing
 - Logging
 - Real-time Telemetry

**You reached the end of the lecture.
Thank you and see you next week.**